

Good — I've now pulled content from every document. This is going to be a long guide, delivered entirely in this chat window. Let's go. I'll work through all five sections methodically.

FIT5225 Cloud Computing & Security — Complete Study Guide

SECTION 1: FOUNDATIONS — THE "WHAT" AND "WHY"

1.1 What is a Distributed System?

Before understanding cloud, you need to understand what cloud is built on top of — **distributed systems**.

Two canonical definitions you must know:

Coulouris: "A system in which hardware or software components located at networked computers communicate and coordinate their actions only by passing messages."

Tanenbaum: "A collection of independent computers that appears to its users as a single coherent system."

These two definitions capture different angles — one focuses on the *mechanism* (message passing), the other on the *illusion* (appears as one system). Both are testable.

Why do distributed systems exist? The core motivation is **resource sharing**:

- **Hardware resources** — disks, printers, scanners
- **Software resources** — files, databases
- **Other** — processing power, memory, bandwidth

The benefits of resource sharing break down into four pillars: **Economy, Reliability, Availability, and Scalability**.

The Three Consequences of Distributed Systems (things that arise naturally from distribution):

1. **Concurrency** — Multiple processes run simultaneously and may conflict when accessing shared resources
2. **No global clock** — Machines cannot perfectly synchronize time; one machine's "now" isn't another's "now"
3. **Independent failures** — Any one node can fail without necessarily knowing about or affecting others

🎯 **Beginner Analogy:** Imagine three friends sharing a Google Doc. They're working simultaneously (concurrency), their laptops' clocks aren't synchronized (no global clock), and one person's laptop dying doesn't stop the others (independent failures). That's a distributed system in everyday life.

1.2 Distributed System Challenges

Quiz Alert — The 8 Distributed System Challenges:

Challenge	What it Means
Heterogeneity	Different hardware, OS, languages must interoperate
Openness	Interfaces are published; systems can be extended
Security	Confidentiality, integrity, and availability must be maintained
Scalability	Must handle growth in users, data, and geography
Failure Handling	Must detect, mask, tolerate, or recover from failures
Concurrency	Shared access to resources must be managed safely
Transparency	Hide the complexity of distribution from users
Quality of Service (QoS)	Meet performance, reliability, and security guarantees

Scalability deserves a deeper look because it's a recurring theme in cloud:

- **Cost of physical resources:** For n users, resources needed should scale as $O(n)$. If one file server supports 20 users, two servers should support 40.
- **Controlling performance loss:** Search algorithms must be scalable — $O(\log n)$ is fine; $O(n^2)$ is not.
- **Resources should not run out:** E.g., 32-bit IP addresses ran out, requiring IPv6 with 128-bit addresses.
- **Avoiding performance bottlenecks:** Decentralized algorithms prevent single-point congestion.

Transparency has multiple types — the goal is hiding the components of a distributed system from both users and programmers. Types include: Access transparency, Location transparency,

Concurrency transparency, Replication transparency, Failure transparency, Mobility transparency, Performance transparency, Scaling transparency.

1.3 What is Cloud Computing?

Cloud computing is the evolution of distributed computing into a commercially delivered utility model.

The "Utility" Origin:

"Distributed resources can be viewed as a commodity or utility similar to water or electricity. Resources are provided by appropriate service suppliers and rented rather than owned by an end user."

This is called **Utility Computing** — the idea that compute power is bought like electricity: pay for what you use, delivered on demand.

Quiz Alert — The Three Official Definitions:

1. UC Berkeley (2009):

Cloud = "data center hardware and software that provide services"

Berkeley characterized cloud computing with three key properties:

1. The **illusion of infinite computing resources**
2. The **elimination of an up-front commitment** by cloud users
3. The **ability to pay for use as needed** (pay-as-you-go)

2. NIST (National Institute of Standards and Technology) — The Most Important Definition:

"Cloud computing is a model for enabling **ubiquitous, convenient, on-demand** network access to a **shared pool** of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be **rapidly provisioned and released** with **minimal management effort** or service provider interaction."

3. Course Definition:

"An IT paradigm that enables access to shared pools of configurable system resources in the form of services that can be rapidly provisioned with minimal management effort, often over the Internet."

🎯 **Beginner Analogy:** Think of electricity. You don't own a power plant to get electricity in your home. You plug into the grid, use what you need, and pay the monthly bill. Cloud computing works the same way — you don't own servers. You plug into AWS/Azure/GCP, use what you need, and pay the bill. The "grid" is the data center.

1.4 The NIST Five Essential Characteristics of Cloud

Quiz Alert — These 5 characteristics appear on almost every cloud quiz. Know them cold.

#	Characteristic	What it Means
1	On-demand self-service	Users can provision compute resources automatically, without human interaction with the provider
2	Broad network access	Resources are available over the network and accessed via standard mechanisms (phones, tablets, laptops, etc.)
3	Resource pooling	Provider's resources serve multiple consumers using a multi-tenant model; physical/virtual resources are dynamically assigned and reassigned
4	Rapid elasticity	Capabilities can be elastically provisioned and released to scale rapidly outward and inward; to consumers, resources appear unlimited
5	Measured service	Resource usage is monitored, controlled, and reported — providing transparency for both provider and consumer. Pay for what you use.

1.5 Why Cloud? — Conventional vs. Cloud Computing

Conventional Computing Problems:

- You must **Buy & Own** hardware, system software, applications
- You buy to meet **peak needs** (wasted during off-peak)
- You must Install, Configure, Test, Verify, Evaluate, and Manage everything
- Hardware refresh cycle: **every ~18 months** (Moore's Law)
- **Very expensive** (capital expenditure)

Cloud Computing Solution:

- Just **Subscribe** and Use
- Pay per **Quality of Service (QoS)**
- No upfront cost; convert CapEx → OpEx

Three Cloud Pricing Models:

- **Tiered pricing** — Fixed specs at fixed prices per unit of time (Amazon EC2 uses this)

- **Per-unit pricing** — Pay per unit of specific resource like RAM/hour (GoGrid uses this)
 - **Subscription-based pricing** — Periodic subscription fee (most SaaS providers)
-

1.6 NIST Three Service Models & Four Deployment Models

This is the NIST framework summary — each component gets a deep section later, but the top-level structure is:

Three Service Models:

1. **SaaS** — Software as a Service
2. **PaaS** — Platform as a Service
3. **IaaS** — Infrastructure as a Service

Four Deployment Models:

1. **Public cloud**
2. **Private cloud**
3. **Community cloud**
4. **Hybrid cloud**

XaaS = "Anything as a Service" — the umbrella term covering all service delivery models.

SECTION 2: INFRASTRUCTURE & ARCHITECTURE — THE "HOW"

2.1 Cloud Architecture Overview

Cloud is a **utility-oriented, Internet-centric way of delivering IT services on demand**, covering the entire computing stack from raw hardware to software services.

The cloud reference architecture can be visualized as layers:

```
[ SaaS Layer ]      ← Applications
[ PaaS Layer ]      ← Development Platforms
[ IaaS Layer ]       ← Virtual Machines / Infrastructure
[ Virtualization ]  ← Hypervisor / VMM
[ Physical Hardware ] ← Servers, Storage, Networking
```

Key insight: Any cloud service — whether virtual hardware, a development platform, or application software — relies on a **distributed infrastructure** owned by the provider (or rented from a third party). In most cases, **hardware resources are virtualized** to provide isolation of workloads and to best exploit the infrastructure.

2.2 Hardware Virtualization — The Core Technology

Virtualization Definition: Virtualization hides the physical characteristics of a computing platform from users, presenting instead an **abstract computing platform**.

🎯 **Beginner Analogy:** Virtualization is like a movie set. The "Wild West" façade looks like a real town from the outside, but behind it is just scaffolding and plywood — fake walls controlled by the director. The guest OS thinks it's running on real hardware, but it's actually running on a fake (virtualized) environment controlled by the hypervisor.

Four Virtualization Features (memorize these):

Feature	What it Provides
Isolation	Fault isolation, performance isolation, software isolation between VMs
Encapsulation	Cleanly captures all VM state; enables snapshots, cloning, and copying of VMs
Portability	VMs are independent of physical hardware; enables live and cold migration
Interposition	Transformations on instructions, memory, I/O; enables transparent encryption, compression, replication

2.3 Key Terminology

Virtual Machine (VM): A representation of a real machine using hardware/software that can host a **guest operating system**.

Guest Operating System: An OS that runs inside a virtual machine.

Host: The original (physical) environment where the guest OS is managed.

Hypervisor / Virtual Machine Monitor (VMM): A thin layer of software on top of the underlying physical machine architecture. It provides the interface that virtual machines interact with — either mimicking hardware or slightly abstracting it.

2.4 Hypervisor Types

Quiz Alert — Type I vs. Type II Hypervisor:

Property	Type I (Native/Bare-Metal)	Type II (Hosted)
Where it runs	Directly on hardware (no host OS)	On top of a host OS
Interface	Exposes ISA to VMs directly	Managed by the OS via ABI
Also called	Native virtual machine	Hosted virtual machine
Performance	Higher (direct hardware access)	Lower (OS overhead)
Examples	VMware ESXi, Xen, Hyper-V	VirtualBox, VMware Workstation

Internal VMM Architecture — Three modules:

- **Dispatcher** — Entry point; routes VM instructions to the right module
- **Allocator** — Decides which system resources to give to each VM when a VM tries to change machine resources
- **Interpreter** — Handles privileged instructions via trap-and-emulate routines

Goldberg & Popek Theorem (1974): For a VMM to efficiently support virtualization, three properties must be satisfied:

1. **Equivalence** — A guest under a VMM behaves the same as on physical hardware
2. **Resource control** — The VMM is in complete control of all virtualized resources
3. **Efficiency** — A statistically dominant fraction of instructions execute without VMM intervention

ISA = Instruction Set Architecture (hardware interface exposed to VMs) **ABI** = Application Binary Interface (OS interface)

2.5 Virtualization Techniques

Full Virtualization

- **Guest OS is completely unaware** it's running in a virtualized environment
- The hypervisor presents a complete emulation of the underlying hardware
- **Broad compatibility** — unmodified guest OSes run transparently
- **Downside** — performance overhead due to hardware emulation (trapping privileged instructions)
- **Examples:** VMware, VirtualBox

Paravirtualization

- **Guest OS is aware** of its virtualized environment
- Communicates with the hypervisor through **special paravirtualized drivers / hypercalls**

- Requires **modifications to the guest OS** (source code must be accessible)
- **Lower overhead** and **improved performance** vs. full virtualization (no hardware emulation needed)
- Introduced in the **Denali project** at the University of Washington; term coined there
- **Example: Xen**

Hardware-Assisted Virtualization

- Hardware itself (CPU) provides architectural support for the VMM
- Introduced by **Intel VT** (formerly Vanderpool) and **AMD V** (formerly Pacifica) — x86 extensions after 2006
- Allows the hypervisor to run in a privileged **Ring -1**, with guest OS in Ring 0
- Eliminates need for binary translation; major performance improvement
- **Examples:** KVM, VirtualBox, Xen (with HW assist), VMware, Hyper-V, Parallels

Partial Virtualization

- **Partial emulation** of the underlying hardware
- Not all OS features supported — some apps run transparently, some don't
- Historical milestone toward full virtualization
- Example: **Address space virtualization** in time-sharing systems (multiple processes in separate memory spaces, sharing disk/CPU/network)

Operating System-Level Virtualization

- Virtualization **within a single OS** — no VMM/hypervisor needed
- The OS kernel creates multiple **isolated user space instances**
- Each instance has its own file system view, IP addresses, software configurations
- Foundation for **containers** (Docker, LXC, etc.)

🎯 Comparison Table — Full vs. Para vs. HW-Assisted:

Property	Full Virtualization	Paravirtualization	HW-Assisted
Guest OS aware?	No	Yes	No
Guest OS modifications?	None	Required	None
Performance overhead	High	Low	Low
Hardware support needed?	No	No	Yes
Example	VMware	Xen	KVM, modern VMware

2.6 Xen — Paravirtualization Case Study

Xen is the most prominent implementation of paravirtualization. Originally developed at the **University of Cambridge**, now backed by a large open-source community. Citrix offers it commercially as **XenSource**.

Xen Architecture Components:

- **Xen Hypervisor (VMM):** Runs in highest privilege (Ring 0 on bare metal; Ring -1 with HW-assist). Handles memory management, CPU state registers, and device I/O.
- **Domain 0 (Management Domain):** A special, privileged VM that runs the control plane. It has access to the Xen hypervisor and provides the HTTP interface and VM management. It is the only domain with direct hardware access.
- **Domain U (User Domains):** Unprivileged guest VMs. Run modified guest OS with hypercalls into the VMM. User applications run with an unmodified ABI.

Xen Rings (x86 Privilege Model):

- Ring 0 → Xen Hypervisor (most privileged)
- Ring 1 → Guest OS
- Ring 3 → User Applications (least privileged)

With hardware-assisted virtualization, Xen runs in **Ring -1**, allowing the guest OS to run in Ring 0 — improving compatibility with unmodified guest OSes.

Xen limitation: Paravirtualization requires modifiable OS source code. Linux is supported natively. Windows typically required hardware-assisted virtualization.

Xen I/O Rings:

- **Front-end driver** — acts as a proxy for the device in the guest domain; accepts commands and communicates with the back-end
- **Back-end driver** — manages multiplexing; lives in Domain 0
- Communication happens via a **shared memory page** (the I/O ring)

2.7 VMware — Full Virtualization Case Study

VMware's technology is based on **full virtualization**, where underlying hardware is replicated and made available to the guest OS — which runs **completely unaware** of the abstraction layer.

VMware Product Stack:

Product	Role
VMware GSX Server	Early server virtualization; daemon-based, remote management

Product	Role
VMware ESX Server	Bare-metal hypervisor; includes modified Linux service console
VMware ESXi Server	Thin, stripped-down bare-metal hypervisor; no service console; smaller footprint
VMware vSphere	Full suite: ESXi + vCenter + networking
VMware vCenter	Centralized management and administration of vSphere installations
VMware vCloud	Turns virtualized data centers into an IaaS cloud; self-provisioning portal
VMware vFabric	PaaS component for scalable Java Web application development
Zimbra	SaaS solution for office automation, email, messaging, collaboration

VMware ESXi Architecture:

- **VMkernel** — Thin POSIX-compliant OS; handles process/thread management, file system, I/O stacks, resource scheduling
- **User World API** — APIs exposing VMkernel functionality to user-space tools
- Each VM has a **VMM (VMM per VM)** for direct virtualization
- Components: Distributed VM File System, Virtual Ethernet Adapter and Switch, Storage Stack, Network Stack, Device Drivers, Resource Scheduling

2.8 Microsoft Hyper-V — Hypervisor Case Study

Hyper-V is Microsoft's infrastructure virtualization solution for server virtualization, shipped as a role within **Windows Server 2008 R2**. It uses a hypervisor-based approach.

Hyper-V Architecture:

- Uses **partitions** for multiple concurrent guest OS execution
- **Root/Parent Partition** — The privileged management partition; hosts the Windows OS and management tools (VMMS, WMI)
- **Child Partitions** — Guest VMs (Windows or Linux); may be "enlightened" (hypervisor-aware) or "unenlightened" (legacy)
- **VMBus** — High-speed communication channel between partitions
- **VSPs (Virtualization Service Providers)** — Live in root partition; handle device I/O
- **VSCs (Virtualization Service Clients)** — Live in child partitions; connect to VSPs via VMBus
- Hypervisor runs in **Ring -1**; kernel in Ring 0; applications in Ring 3

2.9 VM Migration

Cold Migration: Migration of a **turned-off or suspended** VM. The associated disks can be moved from one datastore to another. Simple but causes downtime.

Live Migration: Migration of a **running** VM from one physical host to another **without interrupting** user access or applications. Zero downtime.

Live Migration Use Cases: Hardware maintenance, load balancing and consolidation, energy saving, disaster recovery.

Live Migration — How it Works (Step by Step):

1. Source VMM connects to target VMM
2. Target VMM creates a new guest (new VCPU, etc.)
3. Source sends all **read-only** guest memory pages to target
4. Source sends all RD/WR (dirty) pages to target, marking them clean
5. Source repeats step 4 as pages may be modified (becoming dirty again)
6. When the dirty-page cycle becomes very short, source VMM **freezes** guest, sends VCPU's final state + final dirty pages, tells target to start running
7. Target acknowledges guest is running → source terminates its copy

Live Migration Types:

- **Pre-copy:** Iteratively migrates dirty pages; ensures minimal downtime
- **Post-copy:** Minimal execution state transferred first; pages transferred on-demand during resume time; quicker initial transition
- **Hybrid-copy:** Combines pre-copy and post-copy; balances downtime and data integrity

🎯 **Beginner Analogy (Live Migration):** Imagine you're watching a live TV broadcast that's being played from a studio in Sydney. Live migration is like silently switching the broadcast to a Melbourne studio mid-show — the viewers notice nothing; the stream continues uninterrupted.

Server Consolidation: The practice of aggregating multiple VMs onto a smaller number of fully-utilized physical resources. Uses live migration to move VMs without service disruption.

SECTION 3: SERVICE & DEPLOYMENT MODELS

3.1 IaaS — Infrastructure as a Service

Definition: IaaS delivers **customizable infrastructure on demand** — ranging from single servers to entire infrastructures including network devices, load balancers, database and web servers.

Core Technology: Hardware virtualization — one or more virtual machines are configured and interconnected to define the distributed system.

What the Customer Gets:

- Virtualized hardware and storage
- Network infrastructure (virtual networking, VLANs, load balancers)
- Full customization — in most cases just the OS is pre-installed; customer installs everything else
- Or prepackaged images (LAMP stacks, Web servers, DB servers)

IaaS Reference Architecture (three layers):

1. **Physical Infrastructure** — Datacenters, clusters, desktop/heterogeneous resources, third-party IaaS clouds
2. **Infrastructure Management Software** — Pricing/billing, scheduling, monitoring, QoS, SLA management, VM image repository, VM pool management, provisioning
3. **Web-Based Management Interface** — Web services, portals, REST API

Benefits of IaaS:

- Workload partitioning and application isolation
- Sandboxing (secure execution of third-party apps)
- Hardware tuning
- Reduced administration and maintenance cost
- No capital cost for hardware
- **Eliminates depreciation** — hardware doesn't belong to you; it doesn't depreciate on your books

Examples: Amazon EC2, GoGrid, VMware vCloud, Eucalyptus, OpenNebula

3.2 PaaS — Platform as a Service

Definition: PaaS provides a **development and deployment platform** for running applications in the cloud. It is the middleware on top of which applications are built.

PaaS Reference Architecture:

- **Bottom:** Physical Infrastructure / IaaS Providers

- **Middle:** PaaS Core Middleware — Resources Management, Application Management, User Management, QoS/SLA, Elasticity & Scaling, Runtime
- **Top:** Web-based interface (Web Services, Portals, REST API), plus Programming APIs/Libraries

What PaaS Does:

- Provides programming APIs and frameworks
- Provides deployment systems
- Manages provisioning, scaling, and elasticity automatically
- Cuts cost across development, deployment, and management

Vendor Lock-in Risk in PaaS: PaaS binds applications to the provider's specific runtime. Some solutions like Force.com use proprietary frameworks (hard to migrate). Others like Google AppEngine and Microsoft Azure rely on industry-standard runtimes but with private data storage, creating partial lock-in.

Examples: Google AppEngine, Microsoft Azure, Manjrasoft Aneka, DataSynapse, ElAstra Cloud Server

3.3 SaaS — Software as a Service

Definition: SaaS is a software delivery model that provides access to **applications through the Internet as a Web-based service**. Users access the application via a browser; no installation or local management required.

The "One-to-Many" Model: One application instance is shared across multiple users (multi-tenant). This is the defining characteristic of SaaS.

How It Works:

- Customer visits the application website, enters credentials and billing details
- Immediately uses the application — no setup, no licensing fee
- Provider maintains all infrastructure and customization details per customer

Common SaaS Use Cases: CRM (Customer Relationship Management) and ERP (Enterprise Resource Planning) — needs that are common across almost all enterprises.

Examples: Salesforce.com (CRM), Clarizen.com (project management), Google Apps, Zimbra (by VMware)

SaaS Pricing: Subscription-based — periodic fee for use of software.

3.4 Comparison Table: IaaS vs. PaaS vs. SaaS

Quiz Alert — This table is gold:

Dimension	IaaS	PaaS	SaaS
What is delivered	Virtualized hardware + storage	Development + deployment platform	Ready-to-use applications
Customer manages	OS, runtime, apps, data	Just applications and data	Nothing (provider manages all)
Provider manages	Hardware, virtualization, networking	Hardware + OS + runtime + middleware	Everything
Who uses it	System admins, DevOps	Developers	End users
Examples	Amazon EC2, Eucalyptus	Google AppEngine, Azure	Salesforce, Google Apps
Pricing	Per hour / per unit	Variable	Subscription
Lock-in risk	Low (VMs are portable-ish)	Medium-High (API/runtime dependency)	Low (browser-based)

3.5 Deployment Models

Public Cloud

- Cloud infrastructure is open to the **wider public**
- Shared by multiple tenants (multi-tenant model)
- Provider bears all infrastructure costs
- **Advantages:** Massive scale, no upfront cost, elastic scaling
- **Disadvantages:** Security concerns, less control, compliance challenges
- **Examples:** Amazon Web Services, Google Cloud, Microsoft Azure

Private Cloud

- Cloud implemented **within the private premises** of an institution
- Accessible only to members of the institution (or subset)
- **Use Case:** Organizations with existing IT infrastructure, compliance requirements, or sensitive data
- **Implementation Options:**
 - **VM Technologies:** Xen, KVM, VMware
 - **Infrastructure Management:** Eucalyptus, OpenNebula, VMware vCloud, OpenPEX, InterGrid
 - **PaaS Solutions:** DataSynapse, Zimory Pools, Elastra CloudServer, Aneka
- **Advantages from Forrester Report:**
 - Customer information protection (in-house security)

- Infrastructure that ensures SLAs (clustering, failover, disaster recovery)
 - Compliance with third-party procedures
- **Disadvantage:** Limited ability to scale elastically on demand compared to public clouds

Hybrid / Heterogeneous Cloud

- **Combination** of private cloud augmented with public cloud resources
- Also called **heterogeneous cloud** because of the mixed infrastructure
- Allows enterprises to: exploit existing IT, maintain sensitive data internally, and scale by provisioning external resources temporarily
- **Cloudbursting:** The practice of dynamically provisioning resources from public clouds when private cloud capacity is exceeded — resources are leased temporarily, then released
 - **Note:** "Cloudburst" also informally means failure of a cloud environment due to inability to handle demand spikes
- **Dynamic Provisioning** is the fundamental enabling component of hybrid clouds
- Tools that support hybrid: OpenNebula (integrates Amazon EC2), InterGrid, Aneka, Haizea (scheduler for cost-based allocation)

Community Cloud

- Shared by several organizations that have a **common purpose or interest**
- Multi-administrative domain; may involve different deployment models
- **Candidate sectors:** Media industry (collaborative content production), Healthcare (shared patient data platforms), Energy, Public sector, Scientific research (science clouds)
- **Benefits of community clouds:**
 - **Openness** — Removes vendor dependency; fair competition
 - **Community** — Infrastructure scales by expanding user base
 - **Graceful failures** — No single point of failure (no single provider)
 - **Convenience and control** — Cloud is owned by the community collectively
 - **Environmental sustainability** — Smaller carbon footprint; harnesses underutilized resources

3.6 Comparison: Deployment Models

Dimension	Public	Private	Community	Hybrid
Who can access	General public	Organization only	Member organizations	Mixed
Control	Provider	Organization	Community	Mixed
Cost	Pay-as-you-go	Capital + operational	Shared	Mixed
Security	Provider-managed	Organization-managed	Collectively managed	Mixed

Dimension	Public	Private	Community	Hybrid
Scalability	Highest	Limited	Moderate	High
Examples	AWS, Azure, GCP	Private data center	Science clouds	AWS + private DC

SECTION 4: OPERATIONAL CONCEPTS

4.1 Scalability

Definition: A system is scalable if it can handle growth in the number of users.

Scalability Dimensions in Cloud Middleware:

- **Performance scalability** — System performance doesn't degrade as load increases
- **Size scalability** — System can support more users or resources
- **Load scalability** — System can handle spikes in demand

The cloud middleware manages enormous numbers of resources and users. The challenge is designing **highly scalable and fault-tolerant systems** that are also easy to manage.

4.2 Cloud Economics — CapEx vs. OpEx

Capital Cost (CapEx): One-time expense for purchasing an asset (hardware, software licenses) that contributes to profit over time. IT capital assets — servers, software — are subject to **depreciation** (loss of value over time → reduces profit on books).

Operational Cost (OpEx): Ongoing costs of running a business (electricity, subscriptions, renting).

The Cloud Shift: Cloud converts IT **CapEx** → **OpEx**.

- Buying hardware → Leasing infrastructure (pay per hour)
- Buying software license → Paying a subscription fee

What Cloud Eliminates:

- Depreciation costs (no IT assets to depreciate)
- Software licensing fees (software belongs to provider)

- Carbon footprint costs (relevant in Australia — taxable)
 - Maintenance and administrative staff costs
 - IT support center costs
-

4.3 Open Challenges in Cloud

Quiz Alert — The major open challenges:

1. **Cloud definition** — No single universally agreed-upon formalization (NIST is closest)
2. **Cloud interoperability and standards** — Vendor lock-in is a major strategic barrier; limited cross-vendor API compatibility
3. **Scalability and fault tolerance** — Designing middleware that scales and tolerates failure simultaneously
4. **Security** — Protecting data in multi-tenant, remote environments
5. **Organizational aspects** — Loss of IT staff value, changed relationships within enterprise, lack of local IT expertise for troubleshooting

Vendor Lock-in: When a customer becomes dependent on a specific provider's proprietary formats, APIs, or runtimes. Switching providers becomes costly or technically complex.

Open Virtualization Format (OVF): An attempt at a standard format for packaging/distributing VM images in a platform-independent way. Supported by few vendors; doesn't solve migration of running instances.

4.4 Fault Tolerance and High Availability

Fault tolerance = ability of the system to **continue operating** despite failures. The design principle is that failures are inevitable; design for failure, not against it.

In cloud middleware context: The cloud manages massive infrastructures where individual node failure is expected. The system must self-heal and keep serving requests even when components fail.

SECTION 5: TECHNICAL DEEP-DIVES

5.1 Containers — VMs vs. Containers

History: Containers are not new. Google has been using container technology for years. Solaris Zones, BSD Jails, and LXC have existed for many years. Docker popularized containers for mainstream use.

What are Containers?

- Containers share a single host OS (OS-level virtualization)
- Ultra-portable (own file system, own networking)
- Fast to start (milliseconds vs. minutes for VMs)
- Isolated from each other at the process level

🎯 **Beginner Analogy:** VMs are like separate houses — each has its own plumbing, electrical, foundation. Containers are like apartments in the same building — they share the same foundation (OS), but each unit has its own walls, door, and identity. Apartments start much faster to "move into" than building a whole house.

Quiz Alert — Containers vs. VMs Comparison:

Dimension	VMs	Containers
Weight	Heavyweight	Lightweight
Performance	CPU overhead >10%, Disk I/O >50%	CPU overhead <5%, Disk I/O negligible
OS	Each VM runs its own OS	All containers share the host OS
Virtualization level	Hardware-level	OS-level
Boot time	Minutes	Seconds / Milliseconds
Memory	Allocates required memory	Requires less memory space
Isolation	Fully isolated; more secure	Process-level isolation; possibly less secure
Legacy apps	Low-medium impact	High impact

5.2 Docker

Docker refers to three things:

Term	Meaning
Docker, Inc.	The company
Docker Engine	The container runtime and orchestration technology
Docker Project	The open-source project (now called Moby); licensed under Apache License 2.0 ; written in Go (Golang)

Docker Engine — Main Components:

Component	Function
Docker Client	CLI; communicates with the Docker Daemon
Docker Daemon	Image management, image builds, REST API, authentication, security, core networking, orchestration
containerd	Container lifecycle management (start, stop, pause, remove)
runc	Small, lightweight CLI wrapper for libcontainer; actually creates/runs containers

Two Editions: Enterprise Edition (EE) — paid, with support; Community Edition (CE) — free, for experimentation.

5.3 Docker Images

Docker Image: A read-only template built from a set of instructions written in a **Dockerfile**. Images are the blueprints for containers.

Key Facts:

- Images are built **layer by layer**
- Each layer has its own **256-bit hash** (crypto ID)
- Each image is identified by its **256-bit hash**
- Multiple containers share the same image layers (efficiency)
- A file in a higher layer **obscures** (overrides) the file directly below it — enabling versioning
- If image content changes → associated crypto hash changes
- **Content hashes** and **Distribution hashes** are two types used

Image Sizes (approximate):

- Alpine Linux Docker image: ~4MB
- Ubuntu Docker image: ~110MB
- Microsoft .NET image: >1.7GB

Pulling an image: `$ docker image pull ubuntu:latest`

5.4 Docker Image Registries

Image Registry: Storage and distribution system for named Docker images.

Types:

- **Local registry** — on the same host
- **Docker Hub** — globally shared public registry (<https://hub.docker.com>)
- **Private registry** — on Docker.com for private images

Registry Types by Trust:

- **Official Docker Registry:** Images vetted by Docker Inc.
- **Unofficial Registry:** Images not vetted — use with care

Image Identification:

- Each image has **tags** (e.g., v2, latest)
- Pull command format: `docker image pull <repository>:<tag>` → e.g., `docker image pull mongo:3.3.11`

5.5 Dockerfile

A **Dockerfile** is a plain-text document describing how to build an app into a Docker image.

Example Dockerfile:

```
FROM alpine
LABEL maintainer="email@example.com"
RUN apk add --update nodejs nodejs-npm
COPY . /src
WORKDIR /src
RUN npm install
EXPOSE 8080
ENTRYPOINT ["node", "./app.js"]
```

This creates 4 layers:

- Base layer: FROM alpine
- Layer: RUN apk add npm...
- Layer: COPY ./src
- Layer: RUN npm install...

Build command: `$ docker image build -t web:latest .` The period (.) tells Docker to use the current working directory as build context.

Why use a Dockerfile?

- Creates an **audit log** of how the image was built (maintainability)

- Automates the build process
 - Enables continuous delivery pipelines
 - Cleaner layering of containers
-

5.6 Docker Compose

Docker Compose allows describing and deploying **multi-container (microservice) apps** in a single declarative **YAML file**, deployed with a single command.

- Compose file passes to the `docker-compose` binary
 - Compose deploys it via the **Docker Engine API**
 - Replaces gluing together scripts and long docker commands
-

5.7 Container Orchestration

What is Container Orchestration? A framework for integrating and managing containers **at scale**. It simplifies container management and helps manage availability and scaling.

Features provided by orchestration tools:

- Networking
- Scaling
- Service discovery
- Load balancing
- Health check and self-healing
- Security
- Rolling updates

The Three Major Orchestration Tools:

Tool	Description	Best For
Kubernetes	Open-source, product of CNCF; originally designed by Google	Enterprise-level, complex applications
Apache Mesos	Cluster management tool; container orchestration through Marathon plugin	Largest scale
Docker Swarm	Integrated in Docker container platform	Small/medium clusters

5.8 Docker Swarm

Docker Swarm is Docker's native clustering and orchestration solution.

Two Major Components:

1. **A secure cluster** — Group of Docker nodes managed together
2. **An orchestration engine** — Manages container scheduling and deployment

Node Types:

- **Managers** — Select a leader; leader keeps track of the swarm state; other managers monitor passively and re-elect if leader fails
- **Workers** — Execute containers (tasks/replicas)

Two Modes for Docker Hosts:

- **Single Engine Mode** — Not participating in a swarm
- **Swarm Mode** — Participating in a swarm

Security: Swarm uses **TLS** (Transport Layer Security) to encrypt communications, authenticate nodes, and authorize roles.

Networking: Uses **VXLAN** for virtual overlay networking (nodes in a swarm may not be in the same LAN).

The atomic unit of scheduling in Swarm = Service (not a container).

Service: A higher-level construct that wraps containers with features like scaling, rolling updates, and simple rollbacks. When a container is wrapped in a service, it is called a **task** or **replica**.

Key Docker Swarm Commands:

Command	Action
<code>docker swarm init</code>	Create new swarm; node becomes first manager
<code>docker swarm join-token worker</code>	Get token to join as worker
<code>docker swarm join-token manager</code>	Get token to join as manager
<code>docker node ls</code>	List all nodes in swarm
<code>docker service create</code>	Create new service
<code>docker service ls</code>	List running services
<code>docker service scale</code>	Scale number of replicas up/down
<code>docker service rm</code>	Delete service (no confirmation)

5.9 Raft Consensus Algorithm

Why Raft? Swarm managers use **Raft** to power manager high availability. If one or more managers fail, the survivors keep the swarm running.

What is a Consensus Algorithm? A mechanism that allows a collection of machines to work as a coherent group that can survive the failure of some of its members. It ensures all agents in the system agree on a **single source of truth**, even if some agents fail. It is a fundamental problem in distributed computing and multi-agent systems.

Raft's Goal: Distribute a state machine across a cluster, ensuring each node agrees upon the same series of state transitions.

Time → Terms:

- Time is divided into **terms**. Each term begins with an **election**.
- After a successful election, a **single leader** manages the cluster until end of term.
- Some elections fail → the term ends without a leader.
- Term transitions may be observed at different times on different servers.

Server States (3):

State	Description
Follower	Default state; receives heartbeats from leader
Candidate	Actively requesting votes from peers
Leader	Manages the cluster; sends heartbeats to followers

Leader Election Process (Step by Step):

1. Servers start up as **followers**
2. Remain followers as long as they receive **heartbeats** (empty `AppendEntry()` RPCs from leader)
3. When a follower receives no communication over an **election timeout** (typically **100–500ms**), it assumes the leader has crashed and starts an election
4. The follower with the shortest timeout transitions to **Candidate** state
5. It **increments its current term** counter
6. It **votes for itself**
7. It sends **RequestVote RPCs** to all other servers, retrying until one of:
 - **a) Receives votes from majority → becomes Leader** → sends `AppendEntry` heartbeats to all
 - **b) Receives RPC from a valid leader → returns to Follower state**
 - **c) No one wins (split vote, election timeout) → increments term, starts new election**

State Machine:

Start up → Follower
Follower --[timeout, start election]→ Candidate

```
Candidate --[receive majority votes]→ Leader
Candidate --[discover valid leader/higher term]→ Follower
Leader --[discover higher term]→ Follower
```

Cluster always has: 1 leader, N-1 followers (at steady state).

🌀 **Beginner Analogy:** Think of a board of directors. The Chair (leader) runs the meetings. If the Chair disappears for too long with no word, the board member who's most antsy (shortest timeout) calls a special election. They nominate themselves, ask others to vote. If they get majority support, they become the new Chair and start sending "I'm in charge" memos (heartbeats) to everyone else.

5.10 Kubernetes

Name Origin: Greek word for "governor," "helmsman of a ship," or "captain."

What it is: Open-source container orchestration system, originally designed by **Google**, maintained by CNCF (Cloud Native Computing Foundation).

Two Things Kubernetes Is:

1. A **cluster** for running applications
2. An **orchestrator** of cloud-native microservices apps

How to Run an App on Kubernetes — The Pattern:

1. Write application as small independent services in your preferred language
 2. Package each service in its **own container**
 3. Define a Kubernetes **Pod** to run the containerized service
 4. Deploy Pods via higher-level controllers (Deployments, DaemonSets, StatefulSets, CronJobs)
 5. **POST** the desired state to the cluster; Kubernetes implements it
-

5.11 Kubernetes Architecture

Two Main Components:

Masters (Control Plane)

A collection of system services forming the control plane. In simple setups, all run on a single host. In production, multiple masters are deployed for HA.

Component	Function
API Server	All communication between all components goes through this; RESTful endpoint on port 443
Cluster Store	Persistently stores entire configuration and state of the cluster
Controller Manager	Watches cluster through multiple independent control loops; responds to events
Scheduler	Watches for new work tasks and assigns them to appropriate healthy nodes

Nodes (Workers)

Where application containers run.

Node Components:

- **kubelet** — Agent on each node; communicates with API server and manages pods
- **Container runtime** — Actually runs containers (e.g., Docker)
- **kube-proxy** — Handles networking/load balancing

5.12 Kubernetes API Objects

Pod

Quiz Alert:

- **The atomic unit of scheduling** in Kubernetes
- Kubernetes does NOT directly deploy containers — it deploys **Pods**
- A Pod is a **wrapper** that allows containers to run on a Kubernetes cluster
- "All or nothing" approach — a group of symbiotic containers that must be kept together

Pod Characteristics:

- Pod is considered **running** if all containers are scheduled and running
- The Pod itself doesn't run anything — it's a **sandbox** for hosting containers
- Multiple containers in a Pod **share the same environment**: shared memory, volumes, network stack
- All containers in a Pod share the **same IP address** (the Pod's IP)
- Two containers in the same Pod communicate using ports on **localhost**

"Pod of whales" — As Docker's logo is a whale, a group of containers is called a Pod (like a pod of whales).

Pod Lifecycle: Created → Live → Die. If a Pod dies unexpectedly, Kubernetes starts a new one. The new Pod has a **new ID and a new IP address** (not the same as the old Pod).

Examples of multi-container Pods:

- Service meshes
- Web container supported by a helper container that pulls latest content
- Containers with a tightly coupled log scraper

Deployments

Wrap Pods and add:

- **Self-healing** — Replaces failed Pods automatically
- **Scaling** — Manages desired replica count
- **Updates** — Rolling updates with zero downtime

Services (Kubernetes Service)

Wrap Pods and add:

- **Stable networking** — Provides a reliable name and IP address (unlike Pods which get new IPs after restart)
- **Load balancing** — Distributes requests across multiple Pods behind the Service

🎯 **Example:** An uploader microservice talks to a renderer microservice via a Kubernetes Service. The Service provides a reliable name and IP and load-balances requests across two renderer Pods behind it.

5.13 Kubernetes vs. Docker Swarm — Scalability Comparison

Dimension	Kubernetes	Docker Swarm	Apache Mesos
Scale	Enterprise-level; complex applications	Small to medium clusters	Largest scale; largest deployments
Complexity	Higher	Simpler	Most complex
Maintained by	CNCF (Google origin)	Docker Inc.	Apache
Unit of scheduling	Pod	Service (task/replica)	Task

5.14 Web Services and Service-Oriented Architecture (SOA)

Web Services

Definition: Web services provide an interface to clients to interact with remote servers in a general way. Interactions occur between client and server via **request and reply messages** transmitted over HTTP, where data is represented via **XML or JSON**.

Why Web Services? They provide an infrastructure for a richer and more structured form of interoperability — enabling clients in one organization to interact with servers in another **without human supervision**. They enable **CRUD** operations (Create, Read, Update, Delete) over the Internet.

Historical Origin: Bill Gates introduced **BizTalk in 1999** — "to provide the technologies to allow software in different places, written in different languages and resident on different platforms to connect and interoperate." The two main implementations that followed: **SOAP-based (WS-*)** and **REST**.

SOAP — Simple Object Access Protocol

Quiz Alert:

- SOAP stands for **Simple Object Access Protocol**
 - SOAP is an **XML-based** protocol for exchanging information over a particular protocol (e.g., HTTP) between applications
 - Was popular in the **early 2000s**
 - Provides **service interfaces**
 - Communicates using **request and reply messages** made of SOAP or XML
 - Has an **Interface Definition Language (IDL)** called **WSDL** (Web Service Definition Language)
 - Can be looked up in a **UDDI** (Universal Directory and Discovery Service) registry
 - **Language independent**
 - Supports **synchronous or asynchronous** communication
 - Based on packaging of **single one-way messages**
 - Originally HTTP-based; current version supports SMTP, TCP, UDP, or HTTP
-

REST — Representational State Transfer

Quiz Alert:

- REST originated from **Roy Fielding's PhD dissertation (2000)**

- Clients use **URLs and HTTP operations (GET, PUT, DELETE, POST)** to manipulate resources
- Web services conforming to REST architectural style are called **RESTful Web Services**
- REST is a **set of design criteria**, not a physical architecture
- Allows stateless operations on representations of Web resources via a uniform, predefined interface
- REST reads can be **cached**; SOAP reads cannot

HTTP Operations in REST:

Operation	Action
GET	Retrieve a resource
POST	Create a new resource
PUT	Update an existing resource
DELETE	Remove a resource

REST vs. SOAP Comparison Table

Quiz Alert — This is a prime comparison table:

Property	SOAP	REST
Type	Protocol (highly specified)	Architectural style (flexible)
Approach	Function-driven	Data-driven
Statefulness	Stateless by default; can be stateful	Stateless; no server-side sessions
Messaging format	XML only	XML, JSON, plain text, YAML, HTML
Caching	API calls NOT cached	API calls can be cached
Security	WS-Security + SSL; ACID compliance	SSL and HTTPS
Performance	Requires more power and bandwidth	Requires fewer resources (lightweight)
Standard	Has an official standard	No official standard for RESTful APIs
Documentation	Complex WSDL	Easier to understand
Transfer protocols	SMTP, HTTP, UDP, TCP	HTTP only
Reliability	Guarantees ACID transactions; retry logic built in	Limited to single HTTP transaction; client must retry
Best for	Financial services, enterprise payment apps, high security	Public APIs, social networks, mobile services

Service-Oriented Architecture (SOA)

Definition: A set of design principles whereby distributed systems are developed using sets of **loosely coupled services** that can be dynamically discovered and communicate with each other or are coordinated through **choreography** to provide enhanced services.

Principal means of realizing SOA: Web services (due to the inherent loose coupling).

Mashup: A new service created by combining two or more existing services. SOA enables this approach.

Microservices

What are Microservices?

- An application architecture where every function runs as its own **independent service**
- Services run **inside containers**
- Services communicate over APIs (Web services, RPC, events, etc.)
- **Loosely coupled, reusable, and specialized** components
- A **cloud-native** architectural approach

Benefits of Microservices:

- Update code more easily (deploy one service without touching others)
- Use different tech stacks for different components
- Scale components **independently** (don't scale the whole app for one bottlenecked service)
- More agile, scalable, and resilient

The progression: Monolith → SOA → Microservices (each iteration adds more granularity and independence)

SOA vs. Microservices:

Dimension	SOA	Microservices
Services scope	Larger, enterprise-level services	Smaller, single-function services
Communication	Often ESB (Enterprise Service Bus)	Direct APIs (REST, gRPC, events)
Deployment	Usually full app deployed together	Each service deployed independently
Data	May share databases	Each service has its own database

5.15 Data Formats: XML vs. JSON

Web services use two main data representation formats:

XML (eXtensible Markup Language):

- Verbose, tag-based
- Self-describing
- Used by SOAP exclusively
- Human-readable but heavy

JSON (JavaScript Object Notation):

- Lightweight, key-value format
 - Easier to parse programmatically
 - Used by REST and modern APIs
 - Less verbose than XML
-

5.16 Distributed Systems — Deeper Topics (from Ch7)

System Virtualization in Distributed Systems Context

The textbook frames system virtualization as a key enabling technology:

- **Problem being solved:** How to run multiple OS environments on a single physical host
- **Solution:** A thin VMM layer provides an interface based on the underlying physical architecture
- **Full virtualization advantage:** Existing OS can run unmodified on the VMM
- **Full virtualization challenge:** Hard to realize with satisfactory performance on x86 (x86 didn't meet Popek & Goldberg requirements natively)

Thread Architectures for Servers

Worker Pool Architecture: A fixed pool of worker threads processes requests from a shared queue. I/O thread places requests; worker threads pick them up.

- **Disadvantage:** Fixed pool size may be inadequate for demand spikes; high switching overhead between I/O and worker threads

Thread-per-request Architecture: The I/O thread spawns a new worker thread for each request; thread destroys itself after processing.

- **Advantage:** No contention on shared queue; throughput maximized
- **Disadvantage:** Thread creation/destruction overhead

Thread-per-connection Architecture: One thread per client connection, persisting for the life of the connection.

- Client may make many requests over the same connection
- Thread is destroyed when client closes connection

SECTION 6: ADDITIONAL TECHNICAL TOOLS & PLATFORMS

6.1 OpenNebula, Eucalyptus, and Private Cloud Tools

Eucalyptus: Open-source IaaS management tool with **full compatibility with Amazon Web Services (AWS) interfaces**. Supports KVM, Xen, VMware. Can integrate with public clouds (Amazon EC2).

OpenNebula: Open-source virtual infrastructure management supporting KVM, Xen, VMware. Modular architecture; designed to integrate third-party IaaS providers. Supports hybrid clouds by integrating Amazon EC2 resources.

Haizea: Advanced scheduler that integrates with OpenNebula for **cost-based scheduling** (optimizing budget when renting public VM instances).

InterGrid: Distributed scheduling engine managing VM allocation across peer networks (local clusters, public cloud gateways). Considers user budget and peering arrangements.

Aneka: Software development platform for deploying cloud infrastructure on heterogeneous hardware (datacenters, clusters, desktop grids). Supports bag-of-tasks, MapReduce, and other programming models. Provides a PaaS provisioning service that leverages IaaS providers for scaling.

DataSynapse: Application virtualization software; flexible environment for building private clouds on top of datacenters.

Elastra Cloud Server: Platform for configuring and deploying distributed application infrastructures on clouds.

Zimory: Software infrastructure layer automating resource pools based on Xen, KVM, VMware.

6.2 OpenStack (from Course Syllabus Context)

While detailed OpenStack slides aren't in the uploaded documents, the course covers **OpenStack** as part of the lab work (Nectar uses OpenStack). Key components from context:

- **OpenStack** is the open-source cloud platform used in Australian Research Cloud (Nectar)
 - It implements an IaaS layer with components for compute (Nova), networking (Neutron), storage (Swift, Cinder), identity (Keystone), and image management (Glance)
 - Used in Lab Week 1 as the first hands-on environment
-

6.3 Cloud Security — Open Challenges

Security is listed as both a distributed system challenge and a cloud-specific open challenge. Key concerns:

- **Multi-tenant risk:** Multiple organizations share the same physical infrastructure
- **Data sovereignty:** Political and legal implications of where data is stored and processed
- **Loss of control:** Organizations lose direct control over management of data and processes
- **Organizational disruption:** Existing IT staff must develop different competencies; value of local IT expertise decreases
- **Compliance:** Third-party compliance standards may not be met in public cloud environments

Private clouds are advantageous for security because:

- Customer data stays on-premises
 - Organization controls access and security policies
 - Easier to maintain and verify SLA compliance
 - No dependency on provider's undisclosed security practices
-

6.4 Quality of Service (QoS) and SLAs

QoS (Quality of Service): Specific performance guarantees made to users — including appropriate clustering, failover, data replication, system monitoring, maintenance, and disaster recovery.

SLA (Service Level Agreement): The formal agreement between provider and customer specifying the level of service to be delivered.

In cloud context: Pricing tiers are linked to SLA levels. IaaS solutions offer SLA-based resource allocation as part of the infrastructure management software layer.

MASTER LIST: 20 PRACTICE QUIZ QUESTIONS

Based strictly on the content from your uploaded documents:

- Q1.** According to NIST, what is the official definition of cloud computing, and what are the five essential characteristics it identifies?
- Q2.** What are the three service models in the NIST cloud framework? Briefly describe what each delivers to the customer.
- Q3.** What is the difference between a Type I and Type II hypervisor? Give one example of each.
- Q4.** Compare full virtualization and paravirtualization. Which requires guest OS modification, and which example technologies use each approach?
- Q5.** What are the four virtualization features (Isolation, Encapsulation, Portability, Interposition)? Briefly describe what each one enables.
- Q6.** Explain live migration vs. cold migration of VMs. What are the four use cases for live migration?
- Q7.** What are the three live migration types (Pre-copy, Post-copy, Hybrid-copy), and what is the core difference between them?
- Q8.** What are the three pricing models used by cloud providers? Which model does Amazon EC2 use, and which model does GoGrid use?
- Q9.** What is cloudbursting, and in which type of cloud deployment does it apply? What role does dynamic provisioning play?
- Q10.** What is the Raft consensus algorithm, and what problem does it solve? List the three server states in Raft.
- Q11.** Describe the full Raft leader election process step by step. What is the typical election timeout range?

Q12. What is the fundamental difference between a Docker container and a VM in terms of what they share and their boot time?

Q13. What is a Kubernetes Pod? Why can't you deploy a container directly in Kubernetes without a Pod? What does "all or nothing" mean in Pod context?

Q14. In Kubernetes, what do Deployments add over a basic Pod, and what does a Kubernetes Service add over a Deployment?

Q15. Compare Docker Swarm and Kubernetes in terms of scalability and their atomic units of scheduling.

Q16. What does SOAP stand for, and what is its IDL called? What data format does SOAP exclusively use?

Q17. Where did REST originate, and what HTTP operations does a RESTful Web service use? How does REST's caching behavior differ from SOAP?

Q18. What is a community cloud? List three candidate sectors that would benefit from a community cloud and why.

Q19. What are the four deployment models of cloud computing? For each, in one sentence, describe who has access.

Q20. What are the eight challenges of distributed systems? Which three are described as the natural "consequences" of distributing a system?

It was already indexed in the project knowledge, so the content was accessible. But to be transparent with you — in my original study guide I noted it only briefly with a placeholder because the detailed slides weren't in what I'd pulled at that point. Now I have everything from `5OpenStack.pdf` fully loaded. Here's the complete OpenStack section to add to your study guide:

SECTION 7: OPENSTACK — DEEP DIVE

7.1 What is OpenStack?

OpenStack is a **free and open-source cloud computing software platform**, primarily delivering **IaaS (Infrastructure as a Service)**. It consists of interrelated components (called **services**) that control and support compute, storage, and networking resources.

Key Facts:

- **Started in 2010** as a joint project between **Rackspace** and **NASA**
- Released under the **Apache License**
- Managed and coordinated by the **OpenStack Foundation** — a non-profit corporate entity established in **2012** to promote OpenStack software and its community
- Accessed via: web-based dashboards, command-line tools, or programmatically through **RESTful APIs**

🎯 **Beginner Analogy:** Think of OpenStack as the "operating system for a data center." Just like Windows manages your laptop's CPU, memory, disk and lets you run apps — OpenStack manages an entire data center's servers, storage, and networking, and lets cloud tenants spin up VMs and services on demand.

7.2 Simplified Core Architecture

The six foundational services that form the backbone of any OpenStack deployment are:

```
[ Horizon ]   ← Web Dashboard (user interface)
[ Nova     ]   ← Compute (VMs)
[ Neutron  ]   ← Networking
[ Cinder   ]   ← Block Storage
[ Glance   ]   ← Image Service
[ Swift    ]   ← Object Storage
[ Keystone ]   ← Identity / Auth (underpins everything)
```

Keystone is the spine — every other service registers with it and uses it for authentication.

7.3 Complete Component List

Quiz Alert — Know every code name and its function:

Service	Code Name	Function
Compute	Nova	Provides virtual servers on demand
Image Service	Glance	Catalog and repository for virtual disk images
Block Storage	Cinder	Persistent block storage for guest VMs
Object Storage	Swift	Store/retrieve unstructured files via REST API
Identity/Security	Keystone	Authentication and authorization
Orchestration	Heat	Template-based orchestration for cloud applications
Networking	Neutron	Define, utilize, and consume networking resources

Service	Code Name	Function
Metering/Telemetry	Ceilometer	Infrastructure to collect info/metrics for all OpenStack projects
Database Service	Trove	Database as a Service (relational and non-relational)
Dashboard	Horizon	Modular web-based UI for all OpenStack services
Search	Searchlight	Indexing and search across OpenStack resources
Security API	Barbican	Secure storage and management of secrets (passwords, encryption keys, X.509 certificates)
Data Processing	Sahara	Provision Hadoop, Spark, Storm clusters on demand

7.4 Key Services — In Depth

Nova — Compute Service

What it does: Provides **virtual servers on demand**. The core compute engine of OpenStack.

Key capabilities:

- Supports creating **virtual machines**, **bare-metal servers** (via Ironic), and limited support for system containers
- Responsibilities: **spawning, scheduling, and decommissioning** of virtual machines on demand
- **Virtualization-agnostic** — works with multiple backends:
 - **Libvirt** (open-source API/daemon): supports KVM, QEMU, Xen, LXC
 - XenAPI, Hyper-V, VMware ESX, Docker
- Can **scale horizontally** on standard hardware and download images to launch

Nova Internal Components:

Component	Role
DB	SQL database for data storage
API	Receives HTTP requests, converts commands, communicates with other components
Scheduler	Decides which host gets each instance
Network	Manages IP forwarding, bridges, VLANs
Compute	Manages communication with hypervisor and VMs
Conductor	Handles requests needing coordination; acts as database proxy; handles object conversions

Integration: Nova interacts with Keystone (authentication), Glance (disk/server images), and Horizon (admin interface).

Glance — Image Service

What it does: Discovers, registers, and retrieves **virtual machine images**.

Key capabilities:

- Has a **RESTful API** for querying VM image metadata and retrieving actual images
- Accepts image requests (from Swift) and retrieves/install via Nova
- **Glance-registry** was DEPRECATED in the Queens release

Glance Internal Components:

Component	Role
glance-api	Handles image discovery, retrieval, and storage requests
glance-registry	<i>(Deprecated)</i> — stored, processed, retrieved image metadata
Storage repository	Stores image files: normal file systems, Object Storage, RADOS block devices, VMware datastore
Database	Stores image metadata
Metadata definition service	API for vendors, admins, and users to define custom metadata

Keystone — Identity Service

What it does: Provides **authentication and authorization** for all OpenStack services.

Key capabilities:

- Organized as a group of **internal services** used in combination by the frontend
- Example flow: An authenticate call validates user/project credentials → upon success, creates and returns a **token**
- **Tracks users/permissions**
- Each service **registers with Keystone during install** — so Keystone knows where every service is and who can do what with it

🎯 **Beginner Analogy:** Keystone is the bouncer at the door of an exclusive club. Every other service (Nova, Glance, etc.) is a room inside. You present your credentials to the bouncer (Keystone), you get a wristband (token), and that wristband lets you into the rooms you're authorized to enter.

Swift — Object Storage Service

What it does: Stores and retrieves **arbitrary unstructured data objects** via a RESTful API — things like VM images, backups, media files.

Important distinction: Swift stores and retrieves files but **cannot mount directories like a file server** (it is NOT POSIX).

Key characteristics:

- **Not POSIX** — uses **eventual consistency** (not atomic operations)
- Built for **scale** — optimized for durability, availability, and concurrency
- **Fault tolerant** — uses data replication and scale-out architecture
- Writes objects to **multiple drives**, ensuring data is replicated across a server cluster
- Data persists **until deleted**
- Available from anywhere
- Can be used **with or without Nova/compute**
- **Swift client** — allows users to submit commands to REST API via CLI to configure/connect object storage to VMs

Cinder — Block Storage Service

What it does: Provides **persistent block storage** to virtual machine instances. Unlike Swift (objects), Cinder provides raw storage volumes — like a virtual hard disk attached to a VM.

Key capabilities:

- Supports creation and management of block storage devices
- Interacts with Nova to provide volumes for instances
- Enables management of **volume snapshots** and **volume types**
- **Flexible driver architecture** — can interact with multiple storage solutions

Cinder Internal Components:

Component	Role
cinder-api	Routes requests to cinder-volume
cinder-volume	Interacts with block storage service and scheduler to read/write; flexible driver architecture
cinder-scheduler	Selects optimal storage provider node to create volumes (similar to nova-scheduler)
cinder-backup	Provides backup of any volume type to a backup storage provider

Component	Role
Messaging queue	Routes information between Block Storage processes

🎯 Swift vs. Cinder — The Key Difference:

	Swift (Object Storage)	Cinder (Block Storage)
What it stores	Unstructured files/objects	Raw storage volumes (like a hard drive)
Access method	RESTful API	Mounted to a VM like a disk
Use case	Backups, media, VM images	VM OS disk, database storage
POSIX	No (eventual consistency)	Yes
Analogy	Dropbox	External hard drive

Neutron — Networking Service

What it does: Enables end-users/tenants to **define, utilize, and consume networking resources**. Supports networking of all OpenStack services — mainly interacts with Nova.

Key capabilities:

- Offers an API for users to define **networks and attachments** (switches, routers)
- **Pluggable architecture** — supports multiple networking vendors and technologies

Neutron Internal Components:

Component	Role
neutron-server	Accepts and routes API requests to appropriate plug-ins for action; handles port management (default SSH, VM-specific rules); configures availability zone networking (subnets, DHCP)
Networking plug-ins and agents	Plug and unplug ports, create networks/subnets, provide IP addressing
Messaging queue	Routes information between neutron-server and various agents; database to store networking state

Heat — Orchestration Service

What it does: A **template-driven service** to manage the **lifecycle of applications deployed on OpenStack**. This is OpenStack's equivalent of "Infrastructure as Code."

Key concepts:

- Templates describe infrastructure in **human-readable text files**
- Templates allow creation of most OpenStack resource types: instances, floating IPs, volumes, security groups, users, etc.
- **Stack:** The name for the template + the procedure + the resulting infrastructure created from the template file
- Can be integrated with automation tools: **Chef, Puppet, Ansible**

Heat Orchestration Template (HOT) Structure:

Section	Required?	Purpose
heat_template_version	Mandatory	Specifies which version of Heat the template was written for
description	Optional	Describes the intent of the template to a human
parameters	Optional	Allows user customization during deployment (e.g., key-pair names, image IDs)
resources	Mandatory	Specifications of resources to be created
outputs	Optional	Expected values to be returned after the template is processed

Example HOT snippet:

```
heat_template_version: 2015-04-30
description: Simple template to deploy a single compute instance
parameters:
  key_name:
    type: string
    label: Key Name
  image_id:
    type: string
    label: Image ID
  instance_type:
    type: string
    label: Instance Type
resources:
  my_instance:
    type: OS::Nova::Server
    properties:
      key_name: { get_param: key_name }
      image: { get_param: image_id }
      flavor: { get_param: instance_type }
```

Horizon — Dashboard Service

What it does: Provides a **web-based self-service portal** to interact with all underlying OpenStack services.

Key capabilities:

- Launch instances, assign IP addresses, configure access controls — all from a browser
- Built on **Python/Django** web application framework
- **Requires:** Nova, Keystone, Glance, Neutron (other services optional)

Three Central Dashboards:

1. **User Dashboard** — For regular tenants (Project Tab)
2. **System Dashboard** — For administrators (Admin Tab)
3. **Settings Dashboard** — User preferences and configuration

Identity Tab is also part of Horizon for managing users and projects.

Ceilometer — Telemetry/Metering Service

What it does: Monitors and meters OpenStack services for **billing, benchmarking, scalability, and statistical purposes**. Often called "telemetry information."

Ceilometer Internal Components:

Component	Role
ceilometer-agent-compute	Monitors resource utilization on each compute node
ceilometer-agent-central	Aggregator and analyzer; runs on a central management server
ceilometer-agent-notification	Builds and publishes event and metering data; runs on central management server(s)

Trove — Database Service

What it does: Provides **scalable and reliable Cloud Database as a Service (DBaaS)** functionality for both **relational and non-relational database engines**.

Key capabilities:

- Resource isolation, high performance
- Automates: deployment, configuration, patching, backups, restores, monitoring
- Example: Set up 3 VMs with MySQL, CouchDB, MongoDB
- Uses Glance image service for each DB type

Trove Internal Components:

Component	Role
python-troveclient	CLI that communicates with trove-api

Component	Role
trove-api	RESTful API supporting JSON to provision and manage Trove instances
trove-conductor	Receives messages from guest instances wanting to update host information
trove-taskmanager	Manages complex lifecycle tasks
trove-guestagent	Runs inside DB guest instances

Sahara — Data Processing Service

What it does: Provides users a simple means to **provision Hadoop, Spark, and Storm clusters** by specifying parameters like framework version, cluster topology, and hardware node details.

Key capabilities:

- **Fast provisioning** of data processing clusters for development and QA
- Essentially **Analytics as a Service**
- Managed through a **REST API** with a UI in OpenStack Dashboard
- Supports **addition/removal of worker nodes on demand**

Sahara is why this is relevant to you as a Data Science student — it's the OpenStack service that provisions big data processing frameworks on cloud infrastructure.

Barbican — Security API

What it does: Provides **secure storage, provisioning, and management of secret data** such as passwords, encryption keys, and X.509 Certificates.

Searchlight — Search Service

What it does: Provides **indexing and search capabilities** across OpenStack resources.

7.5 NeCTAR Research Cloud

NeCTAR (National eResearch Collaboration Tools and Resources) is Australia's national research cloud, and it's the platform used in FIT5225 labs.

Key Facts:

- Provides **flexible, scalable computing** to all Australian researchers
- **Based on OpenStack framework**
- Supports **thousands of virtual machines** using more than **30,000 VCPUs** across Australia
- Provides access via **projects**, each with an **allocation of time and capacity** (virtual CPUs)

NeCTAR Services (OpenStack-powered):

NeCTAR Service	Powered By
Nectar Database Service (DBaaS)	OpenStack Trove
Nectar Shared Filesystem Service	OpenStack Manila
Nectar Data Processing Service	OpenStack Sahara
Nectar DNS Service	OpenStack Designate

Different NeCTAR sites work together through a **Service Availability Map** to provide a common interface — some services are only available at particular sites.

7.6 OpenStack — Master Cheat Sheet Table

Quiz Alert — The most likely tested facts:

Code Name	Service	One-Line Function
Nova	Compute	Spawns, schedules, decommissions VMs
Glance	Image	Stores and serves VM disk images
Keystone	Identity	Auth and authorization for all services
Swift	Object Storage	Stores unstructured data via REST API (eventual consistency)
Cinder	Block Storage	Persistent block volumes attached to VMs
Neutron	Networking	Defines and manages virtual networks
Heat	Orchestration	Template-driven lifecycle management (HOT templates)
Ceilometer	Telemetry	Meters all services for billing and monitoring
Trove	Database	DBaaS for relational and non-relational databases
Horizon	Dashboard	Web UI for all OpenStack services (Django-based)
Sahara	Data Processing	Provisions Hadoop/Spark/Storm clusters
Barbican	Security API	Manages secrets, encryption keys, X.509 certs
Searchlight	Search	Indexing and search across resources
